

Generics

Method Overloading

<pre>void swap(int n1,int n2){ int temp = n1; n1 = n2; n2 = temp; }</pre>	<pre>void swap(double n1,double n2){ double temp = n1; n1 = n2; n2 = temp; }</pre>	<pre>void swap(Employee n1,Employee n2){ Employee temp = n1; n1 = n2; n2 = temp; }</pre>
---	--	--

<pre>int main(){ swap(10,20) swap(11.22,33.44) swap(e1,e2) }</pre>	Generic code - The code that we write can be work for any type of data is called as generic code - All the data structures and their algorithms can be created as Generic
--	--

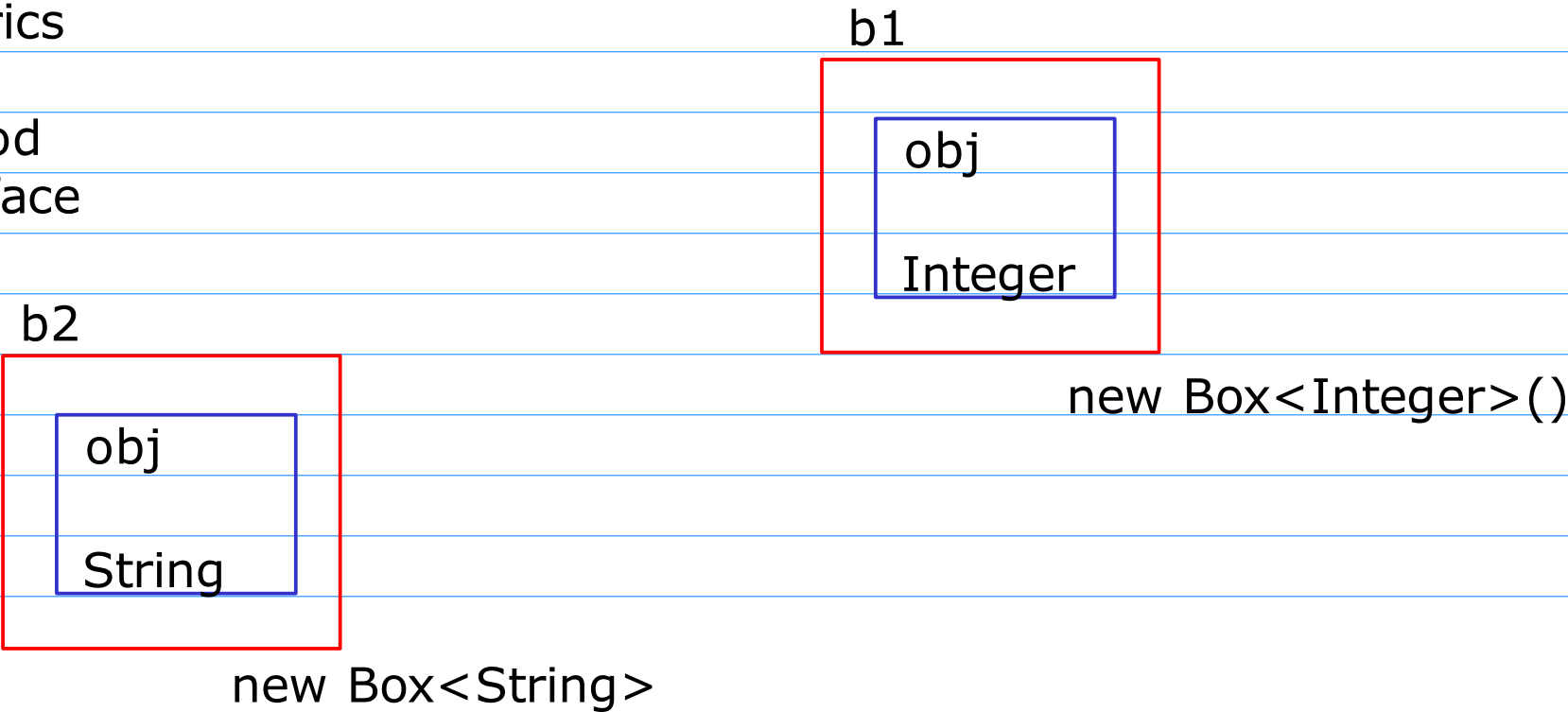
Till java 1.4	From java 1.5 (generics)
---------------	--------------------------

<pre>void swap(Object n1,Object n2){ Object temp = n1; n1 = n2; n2 = temp; }</pre>	<pre><T>void swap(T n1,T n2){ T temp = n1; n1 = n2; n2 = temp; }</pre>
--	--

<pre>int main(){ swap(new Integer(10),new Integer(20)) swap(new Double(11.22),new Double(33.44)) swap(new Employee(1),new Employee(2)) swap(new Employee(1),new Date()) // }</pre>	<pre>int main(){ swap<Integer>(new Integer(10),new Integer(20)) swap<Double>(new Double(11.22),new Double(33.44)) swap<Employee>(new Employee(1),new Employee(2)) swap<Employee>(new Employee(1),new Date()) // }</pre>
--	---

No Type safety	Type-safety
----------------	-------------

- # Generics
1. Class
 2. Method
 3. Interface



```
Box<T>{  
T obj;  
}
```

```
class Person{  
String name;  
}
```

```
class Student extends Person{  
int rollno;  
double marks;  
}
```

```
class Employee extends Person{  
int empid;  
double salary;  
}
```

```
class Manager extends Employee{  
double bonus;  
}
```

```
class SalesManager extends Manager{  
double commission;  
}
```

// unbounded type - upper bound

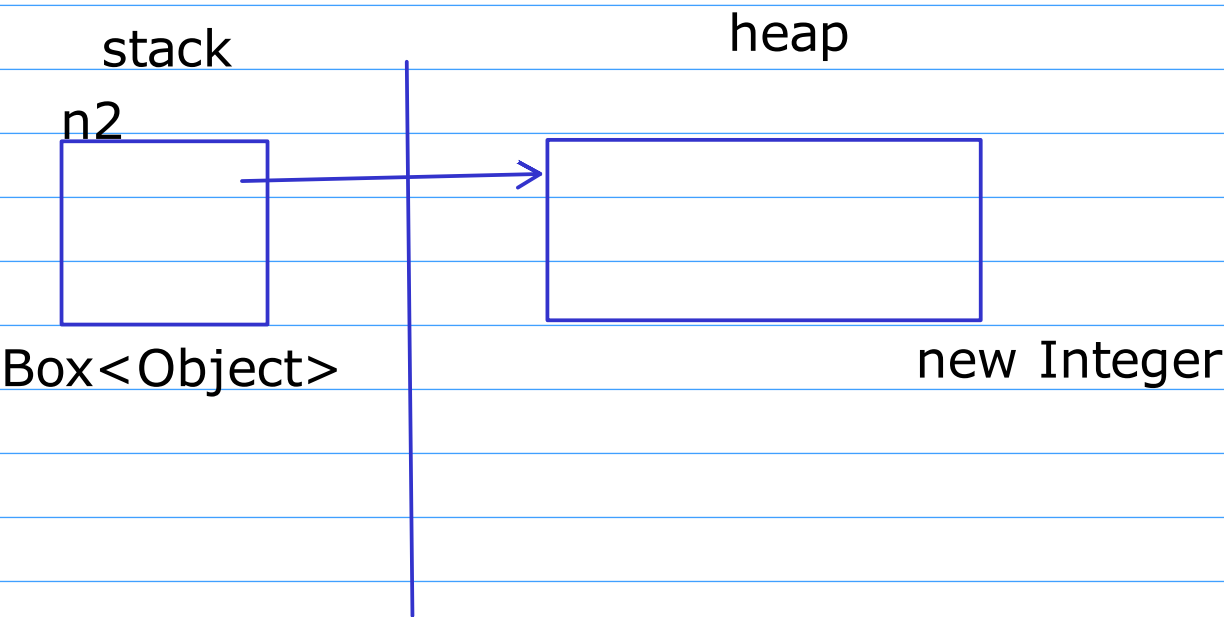
```
Box<? extends Employee>b1 = new Box<Manager>();  
b1.calculateTotalSalary();  
b1.calculateTax();
```

```
Box<? extends Person>b2 = new Box<Student>();  
b1.accept();  
b1.display();
```

// lower bound

```
Box<? super Manager> b1 = new Box<Person>();
```

```
Box<Object> b3 = new Box<String>()  
Box<Number> b3 = new Box<Integer>()
```

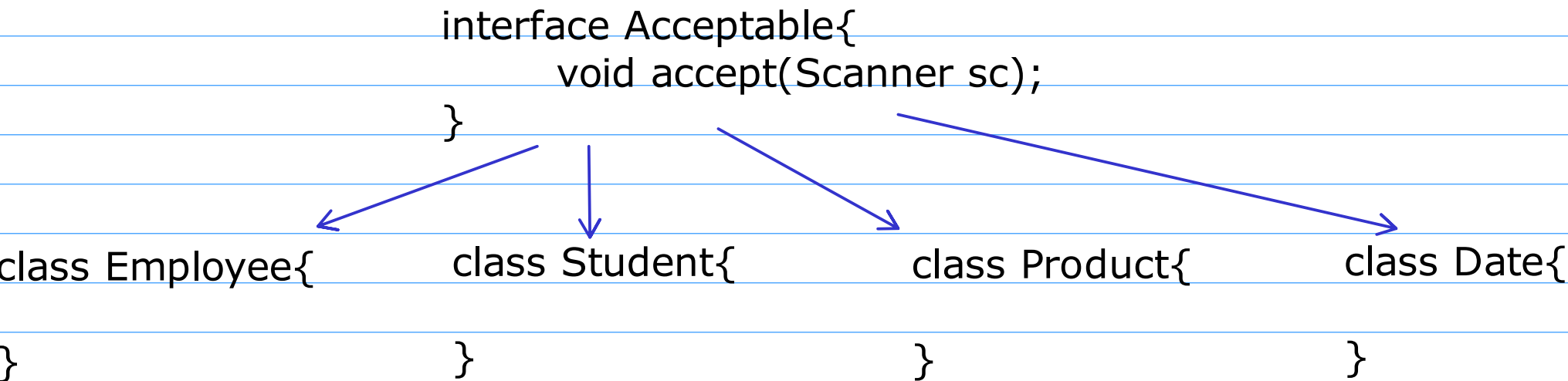


```
String n2 = new String("sunbeam")  
Date n2 = new Date()
```

<pre>class Box { private Object obj; }</pre>	<pre>class Box{ private String obj; }</pre>
<pre>Box<Object> b3</pre>	<pre>new Box<String>();</pre>

<pre>public static void display(Box b1) { }</pre>	<pre>class Box{ // type erasure private Object obj; public void setObj(Object obj) { this.obj = obj; } public Object getObj() { return obj; } }</pre>
.class will not have information about the generic type	

Generic Interface	
till java 1.4	from java 1.5



Generic Interface
till java 1.4

```
interface Comparabale{  
    int compareTo(Object obj)  
}
```

```
class Employee implements Comparable{  
    int compareTo(Object obj){  
        Employee e2 = (Employee) obj;  
        this.salary > e2.salary  
    }  
}
```

```
class Product implements Comparabale{  
    int compareTo(Object obj){  
        Product p2 = (Product) obj; // Downcasting  
        this.price > p2.price  
    }  
}
```

```
Employee e1 = new Employee(10000);  
Employee e2 = new Employee(20000);
```

```
Product p1 = new Product(600);  
Product p2 = new Product(400);
```

e1.compareTo(e2) ✓

p1.compareTo(p2) ✓

e1.compareTo(p2)

from java 1.5

```
interface Comparable<T>{  
    int compareTo(T obj)  
}
```

```
class Employee implements Comparable<Employee>{  
    int compareTo(Employee obj){  
        this.salary > obj.salary  
    }  
}
```

```
class Product implements Comparabale<Product>{  
    int compareTo(Product obj){  
        this.price > obj.price  
    }  
}
```

```
Employee e1 = new Employee(10000);  
Employee e2 = new Employee(20000);
```

```
Product p1 = new Product(600);  
Product p2 = new Product(400);
```

e1.compareTo(e2) ✓

p1.compareTo(p2) ✓

e1.compareTo(p2) // compile time error

Type safety